

Aufgabe 2: Schwierigkeiten

Team-ID: 00233

Team: Bougainvillea

Bearbeiter/-innen dieser Aufgabe:

Tao Zheng

29. Oktober 2024

Inhaltsverzeichnis

1 Lösungsidee: Ziele	1
1.1 Basisveränderungswert und Faktormultiplikator	2
1.2 Einlesung der vergangenen Prüfungen	2
1.3 Die endgültige Reihenfolge	4
2 Umsetzung: Mathematische Funktionen	4
2.1 Main Funktion	5
3 Beispiele	6
3.1 Schwierigkeit 0:	6
3.2 Schwierigkeit 1:	7
3.3 Schwierigkeit 2:	8
3.4 Schwierigkeit 3:	9
3.5 Schwierigkeit 4:	10
3.6 Schwierigkeit 5:	11
4 Quellcode (Python 3.11.9)	12

1 Lösungsidee: Ziele

Zur Lösung der Aufgabe habe ich eine Wertungsskala eingeführt. Die Wertungsskala beschreibt die Schwierigkeit einer Aufgabe in rationalen Zahlen. Jede Aufgabe erhält eine eigene Wertung („Schwierigkeitswert“), die auf den vergangenen Prüfungen basiert. Je kleiner die Zahl, desto einfacher ist die Aufgabe, und je größer die Zahl, desto schwerer ist die Aufgabe. Der Mittelpunkt der Skala ist dabei Null. Dabei möchte ich durch diese Wertungsskala folgende Ziele erreichen.:

1. Stärkerer Einfluss von Prüfungen mit mehreren Aufgaben: Prüfungen, die mehrere Aufgaben beinhalten, sollen stärker gewichtet werden. Das heißt, dass es einfacher sein soll, vom Mittelpunkt zu distanzieren.
 - Prüfungen mit mehreren Aufgaben haben mehr Vergleiche, womit man die Schwierigkeit der Aufgaben besser einschätzen kann.
2. Wiederholte Ergebnisse sollen den Wert stabilisieren: Vergleiche die mehrmals die Richtigkeit einer Schwierigkeitsbewertung bestätigen, sollen einen stärkeren Einfluss auf die Wertung haben und dadurch vor Ausnahmen schützen.
 - Beispiel: Laut 3 unterschiedlichen Klausuren ist Aufgabe A einfacher als Aufgabe B ($A < B$). Jedoch gibt es eine Klausur, in der Aufgabe B einfacher als Aufgabe A war ($B < A$). Aufgrund der 3 vorherigen Klausuren bleibt in der Wertungsskala A einfacher als B, da es sich hierbei um einen Ausnahmefall handelt.
3. Neuere Prüfungen sollen einen etwas größeren Einfluss haben als ältere Prüfungen
 - Falls es in zwei unterschiedlichen Prüfungen zu widersprüchlichen Aussagen kommt, soll die neuere Prüfung stärker gewichtet werden.

- Dies soll erreicht werden, indem man einen „Erwartungsmodifikator“ einbaut.

1.1 Basisveränderungswert und Faktormultiplikator

Als Basisveränderungswert wird eine Zahl verwendet, welche als Standardwert 0.5 hat, und je größer die Distanz ist, desto kleiner wird der Basisveränderungswert (bis zu 0.1).

- Das heißt, dass jede Gleichung den Schwierigkeitswert um mind. 0.1 verändert.
- Dieser Wert wird auf beide Aufgaben einer Gleichung angewendet.

Zusätzlich gibt es den Faktormultiplikator, welcher den Basiswert je nach Situation verändert. Als Standardwert für den Faktormultiplikator wird die Zahl 1 verwendet und wird von den Erwartungsmodifikatoren beeinflusst.:

1. Erwartungsmodifikator: Gleichungen, die von der Einschätzung der Wertungsskala abweichen, bekommen einen Faktormultiplikationsbonus von +20%.

- Beispiel: Laut der Wertungsskala ist Aufgabe A leichter als Aufgabe B ($A < B$). Jedoch ist laut Klausur X die Aufgabe B einfacher als Aufgabe A ($B < A$)
-> Beide erhalten einen Faktormultiplikationsbonus, um die Aktualität aufrecht zu erhalten.
- Diese Anpassung gilt nur, wenn die Distanz der Schwierigkeitswerte der beiden Aufgaben mindestens 0,5 beträgt. Werte unter 0,5 gelten als nicht ausreichend eindeutig, da der normale Basisveränderungswert bei 0,5 liegt.
- Faktormultiplikationsbonus: +0.2
- Zusätzlich wird der Basisveränderungswert von Aufgaben, die nicht der Erwartung entsprechen anders berechnet. In diesem Fall gilt: Je größer die Distanz, desto größer wird der Basisveränderungswert. (jedoch maximal 0.8)

(Kommentar: Es waren ursprünglich weitere Modifikatoren geplant, jedoch haben diese sich als nicht nötig erwiesen. Die Struktur ist weiterhin geblieben, für den Fall, dass ich noch weitere Modifikatoren hinzufüge. Da ich mit meiner Umsetzung zufrieden war, habe ich keine weiteren mehr hinzugefügt und es so gelassen.)

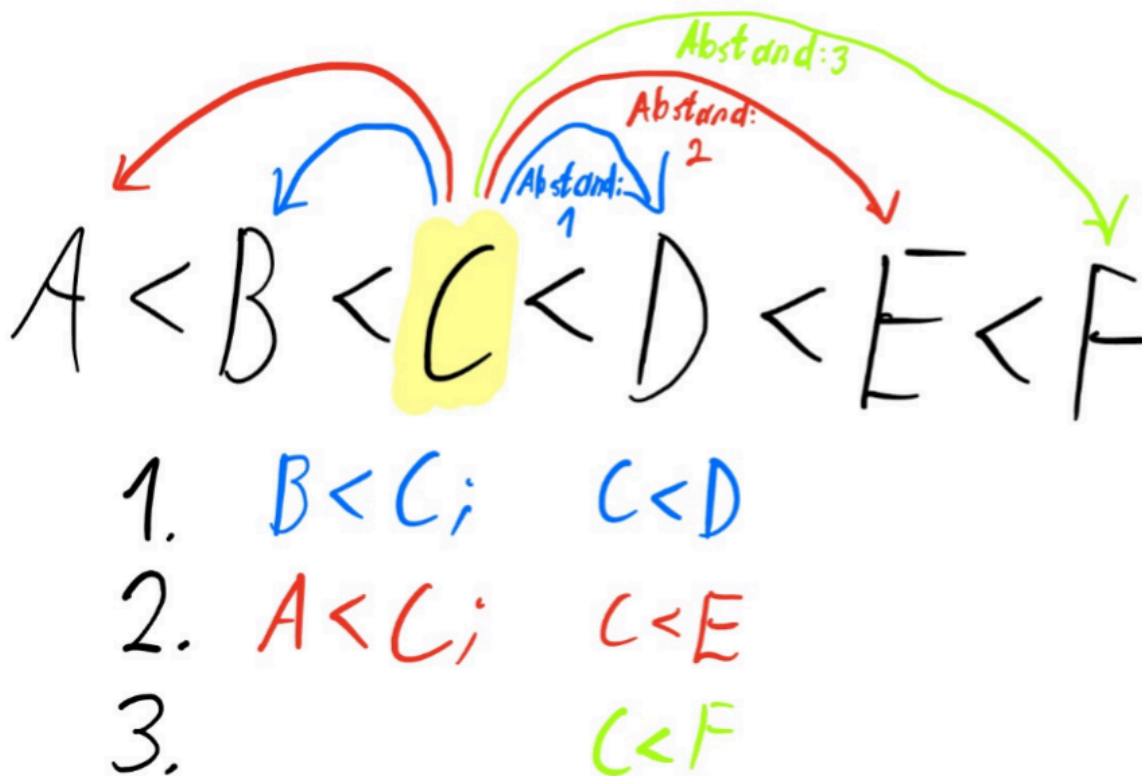
Der endgültige Veränderungswert wird mit folgender Formel ausgerechnet.:

$$\text{Veränderungswert} = \text{Faktormultiplikationsbonus} \cdot \text{Basisveränderungswert} \quad (1)$$

Dieser Veränderungswert wird am Ende zu den Schwierigkeitswerten beider Aufgaben addiert. Alle Veränderungen werden außerdem als Gruppe zusammengefasst und addiert. Es werden keine einzelnen Veränderungen durchgeführt. Die einzelnen Gruppen werden durch den Abstand der Aufgaben in der Gleichung aufgeteilt.

1.2 Einlesung der vergangenen Prüfungen

Man schaut sich zuerst alle nebeneinanderliegenden Gleichungen an und berechnet die Veränderungswerte. Die Veränderungswerte werden bei der einfacheren Aufgabe subtrahiert und bei der schwierigeren Aufgabe addiert. Die Gleichungen verwenden dabei alle die gleichen Schwierigkeitswerte. Sobald man alle nebeneinanderliegenden Gleichungen berechnet sind, werden die Veränderungswerte zu den Schwierigkeitswerten der Aufgaben addiert und man verwendet für die darauffolgenden Gleichungen die aktualisierten Schwierigkeitswerte. Die nächste Stufe der Berechnung beinhaltet Gleichungen mit Aufgaben, die einen Abstand von zwei Aufgaben haben (z.B.: bei $A < B < C$ wird nun $A < C$ betrachtet). Dabei wiederholt man die zuvor beschriebenen Schritte zur Berechnung der Veränderungswerte. Später berechnet man alle Gleichungen, die über 3 Aufgaben, 4 Aufgaben, usw. entfernt sind. Dieser Prozess wird so lange wiederholt, bis keine weiteren Gleichungen mehr möglich sind.



z.B.: $A < B < C < D$

- Gleichungen mit nebeneinanderliegenden Aufgaben werden angeschaut: $A < B$, $B < C$, $C < D$
 - Die Veränderungswerte, die aus diesen Gleichungen berechnet werden, haben noch keinen Einfluss auf den aktuellen Durchgang, sondern wirken erst beim nächsten Durchgang als aktualisierte Schwierigkeitswerte.
- Gleichungen mit Aufgaben, die über 2 Aufgaben entfernt sind, werden angeschaut: $A < C$, $B < D$
 - Aufgrund der zuvor aktualisierten Schwierigkeitswerte sind die Werte von A und D weiter von der Mitte entfernt. Dadurch ist die absolute Distanz zwischen A und C sowie zwischen B und D größer. Dies führt zu geringeren Veränderungswerten im Vergleich zur ersten Runde, die aber weiterhin Einfluss haben.
- Zum Schluss werden alle Gleichungen mit Aufgaben angeschaut, die über 3 Aufgaben entfernt sind.: $A < D$
 - Durch die Beeinflussung aller vorherigen Gleichungen ist die absolute Distanz von A und D noch größer geworden. Der Veränderungswert ist dadurch noch kleiner.
 - Nach diesem Durchgang sind keine weiteren Gleichungen mehr möglich und die endgültigen Schwierigkeitswerte werden für die nächsten Prüfungen gespeichert.

Dadurch soll eines der Ziele erreicht werden, dass Prüfungen mit mehreren Aufgaben stärker bewertet werden.

- Gäbe es beispielsweise für die Aufgabe A in der Prüfung nur die Aufgaben A und B, wobei $A < B$ gilt, würde sich der Schwierigkeitswert nur einmal durch $A < B$ verändern.

- Wenn man es mit dem vorherigen Beispiel vergleicht, würde A die Veränderungswerte von: $A < B$, $A < C$ und $A < D$ (in der Reihenfolge) erhalten, wodurch der Gesamtveränderungswert für A mit mehreren Aufgaben größer ist, als bei Prüfungen mit nur einem Vergleich.

1.3 Die endgültige Reihenfolge

Sobald alle ehemaligen Prüfungen durchgegangen sind, werden die gewünschten Aufgaben eingelesen und nach dem Schwierigkeitswert der Aufgaben sortiert. Nach der Sortierung werden alle gewünschten Aufgaben in der optimalen Reihenfolge ausgegeben und mit einem „kleiner als“ Zeichen getrennt. Man kann nun diese Reihenfolge für die nächsten Prüfungen verwenden. Gibt es eine Sondersituation, wo zwei Aufgaben den gleichen Schwierigkeitswert haben, werden die Aufgaben mit einem „ist gleich“ Zeichen getrennt und der Lehrer kann selbst entscheiden, welche Aufgabe er bevorzugt.

2 Umsetzung: Mathematische Funktionen

Für die Umsetzung habe ich das Programm in Python geschrieben. Um die Ergebnisse zu visualisieren, wurde matplotlib.pyplot verwendet. Matplotlib ist ein Modul und ermöglicht es, Daten grafisch darzustellen (z.B. in Form von Punktdiagrammen mit x und y-Achsen). Diese Diagramme werden verwendet, um die Ergebnisse zu visualisieren.

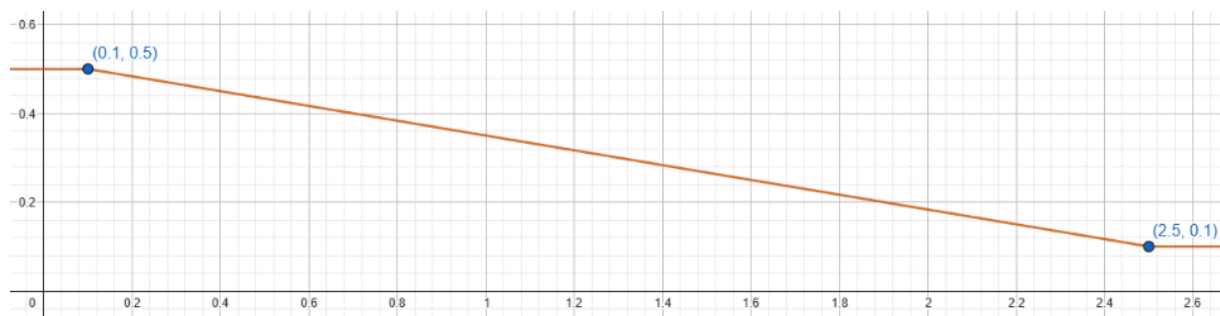
Am Anfang habe ich die Funktion „mathfunction(int)“ erstellt, die eine mathematische Funktion $f(x)$ darstellt. Die Funktion verwendet die absolute Distanz zwischen zwei Werten als Argument und ist mathematisch wie folgt definiert (Funktion f , x =Differenzbetrag, y =Basisveränderungswert):

$$y = 0.5 \text{ für } x \leq 0.1 \quad (2)$$

$$y = 0.5 \cdot \left(-\frac{1}{3} \cdot (x - 0.1) + 1 \right) \text{ für } 0.1 < x < 2.5 \quad (3)$$

$$y = 0.1 \text{ für } x \geq 2.5 \quad (4)$$

Als Rückgabe erhält man eine Zahl, die die absolute Veränderung des Wertes darstellt. Im Folgenden wird die Funktion grafisch dargestellt.:



(x: absolute Differenz, y: Basisveränderungswert)

Zusätzlich gibt es die Funktion „diff“, die als Argumente den Schwierigkeitswert von den einfacheren und schwierigeren Aufgaben aufnimmt. In der Funktion wird der Basisveränderungswert mithilfe der Differenz zwischen den beiden Aufgaben durch mathfunction ausgerechnet.

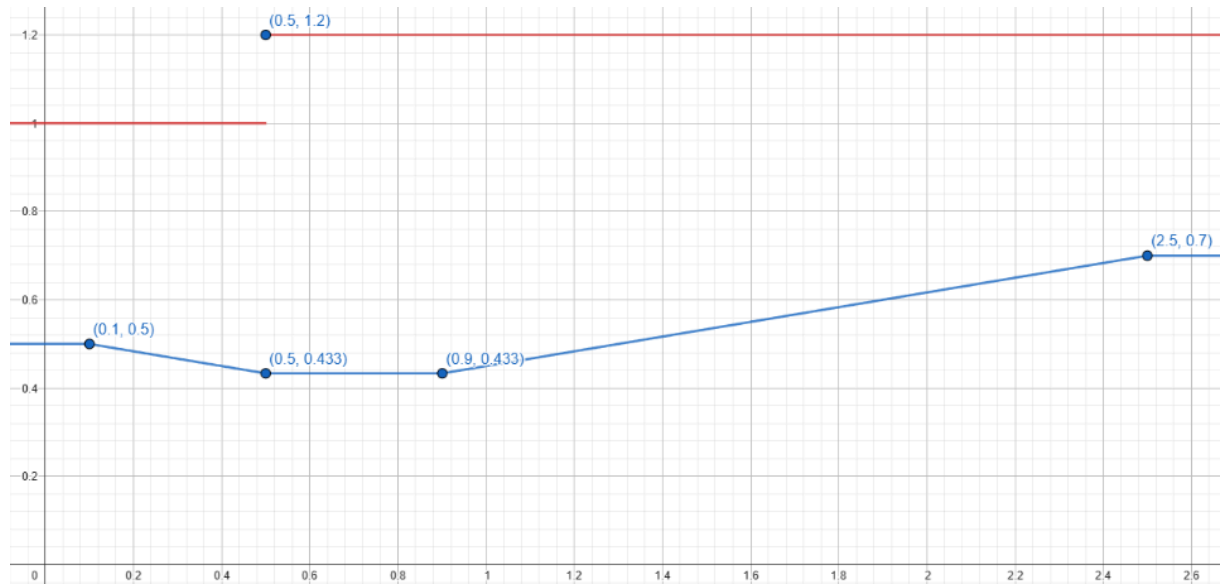
Anschließend wird geschaut, ob die Gleichung nicht erwartet wurde (absolute Differenz von beiden Aufgaben > 0.5 und in der Gleichung ergibt eine falsche Aussage).

- Wenn es erwartet wurde, wird nur der Basisveränderungswert zurückgegeben (da Faktormultiplikationsbonus = 1 ist).
- Ist diese Gleichung nicht erwartet worden, so verwendet man einen Faktormultiplikationsbonus von 1.2 (+20%) und der Basisveränderungswert wird folgendermaßen ausgerechnet ($f(x)$ ist der Rückgabewert von mathfunction()):

$$y = \frac{1.3}{3} \text{ für } x < 0.9 \quad (5)$$

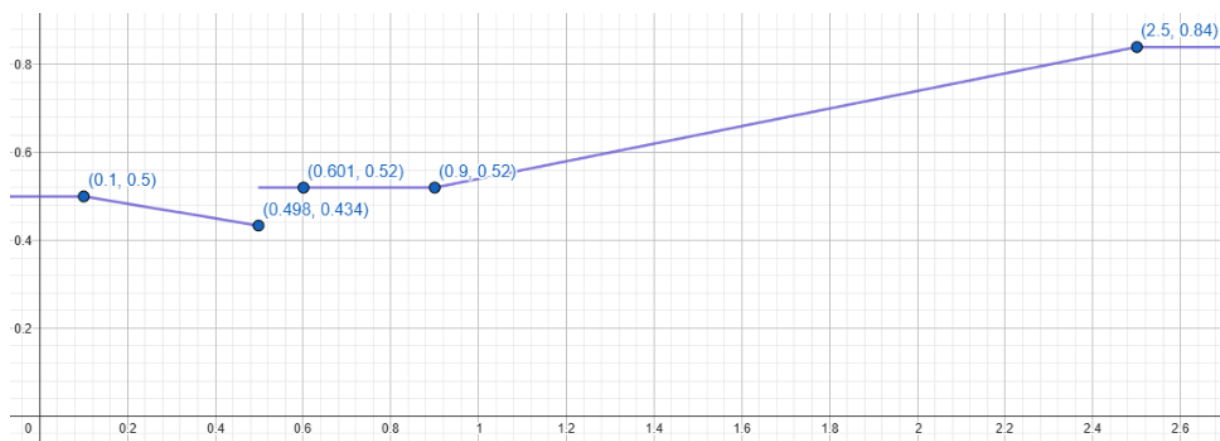
$$y = 0.8 - f(x) \text{ für } x \geq 0.9 \quad (6)$$

Der endgültige Wert wird durch die Multiplikation des Basisveränderungswerts mit dem Faktormultiplikationsbonus berechnet und anschließend zurückgegeben. Im Bild wurden beide Funktionen dargestellt:



(Fall: In der Gleichung ergibt sich eine falsche Aussage, x: Differenz, Graph Rot: Faktormultiplikationsbonus, Graph Blau: Basisveränderungswert)

Tatsächlicher Rückgabewert (mit Faktormultiplikationsbonus * Basisveränderungswert) bei falscher Aussage:



2.1 Main Funktion

Nun kommen wir zur Hauptfunktion „main“. Diese Funktion nimmt das in der Readme.txt Datei angegebene Format an und druckt die optimale Reihenfolge aus. Alle Aufgaben bekommen im Dictionary: tasks die Schwierigkeitswert von 0.0 zugewiesen und man geht anschließend durch alle Prüfungen durch. Die vergangenen Prüfungen werden als eine Liste aus Prüfungen dargestellt und eine Prüfung ist eine Liste aus Aufgaben, welche mit den einfachsten Aufgaben beginnt und mit den schwersten endet. (z.B.: Prüfungen = [[„A“, „B“], [„D“, „C“]]). Man geht durch alle einzelnen Prüfungen durch und macht für jede Prüfung eine Kopie von Tasks (pendingTasksUpdates). Man geht mit allen möglichen Abständen, angefangen mit 1, durch alle Gleichungen durch. Dabei rechnet man mithilfe von der Funktion „diff“ die Veränderungswerte für jede Gleichung aus und addiert es bei der schwierigeren Aufgabe und

subtrahiert bei der einfacheren Aufgabe im `pendingTasksUpdates`. Sobald alle Gleichungen für einen Abstand ausgerechnet wurden, ist `pendingTasksUpdates` der neue `tasks` und man übernimmt dadurch alle Veränderungen, die für die darauffolgenden Abstände gültig ist.

Sobald alle Prüfungen durch sind, werden alle Aufgaben in der Reihenfolge abhängig vom Schwierigkeitswert sortiert und die gewünschten Aufgaben werden ausgedruckt.

Beim Ausdrucken werden alle Aufgaben mit einem „kleiner als“ Zeichen getrennt außer wenn zwei Aufgaben die gleichen Schwierigkeitswert haben. In diesem Fall wird es mit einem „ist gleich“ Zeichen getrennt und der tatsächliche Schwierigkeitswert wird auch noch ausgedruckt.

3 Beispiele

3.1 Schwierigkeit 0:

Die Lösung ist das Gleiche, wie die im Beispiel angegeben wurde. Zum Thema Konflikt hat sich bei mir $G < F$ durchgesetzt. Das liegt daran, dass in Klausur 1 mehrere Gleichungen verwendet wurden, wobei F dabei war und in der letzten Klausur $G < F$ gilt. Dies kann man in der Grafik erkennen ($y =$ Schwierigkeitswert), die ich am Ende jeder Aufgabe hinzugefügt habe.

$B < E < D < F < C$

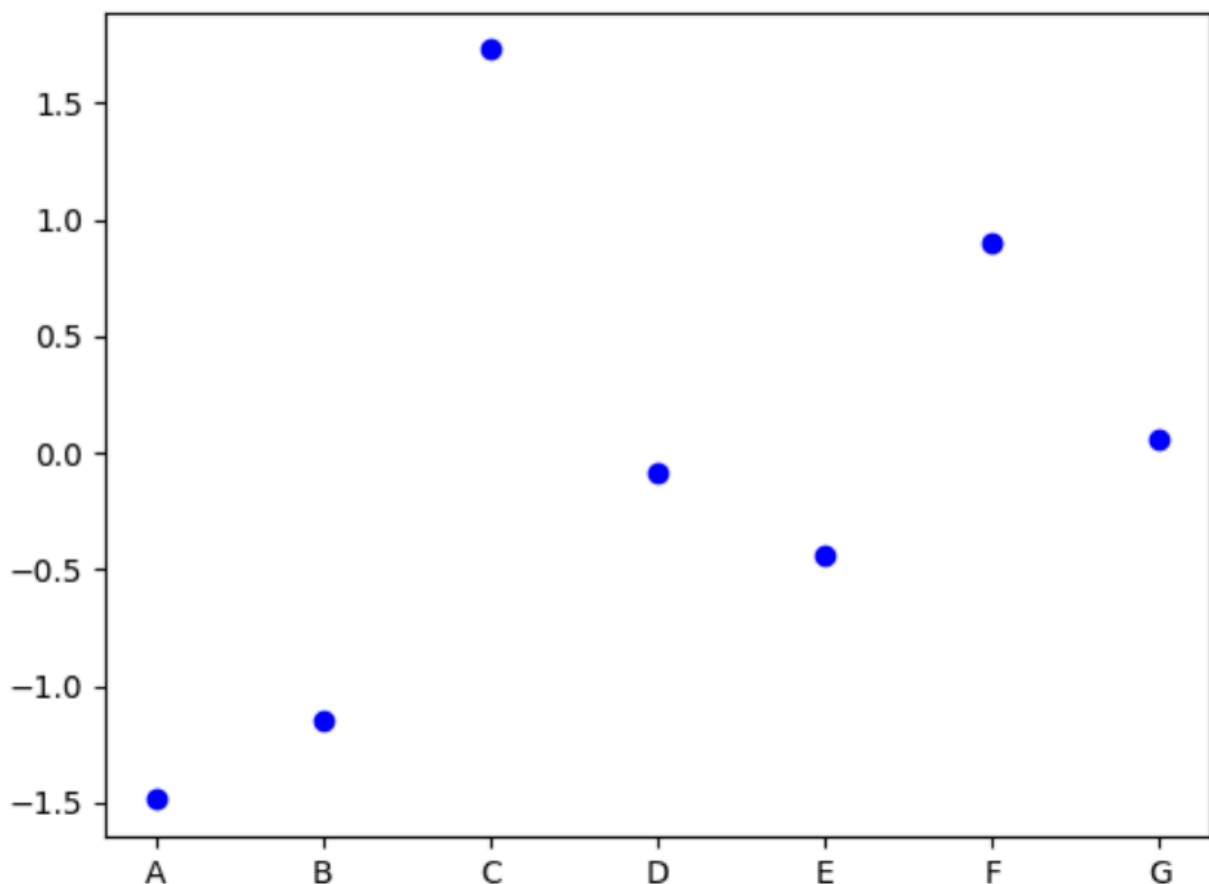
B: -1.150925925925926

E: -0.43960152796829755

D: -0.08768692367779302

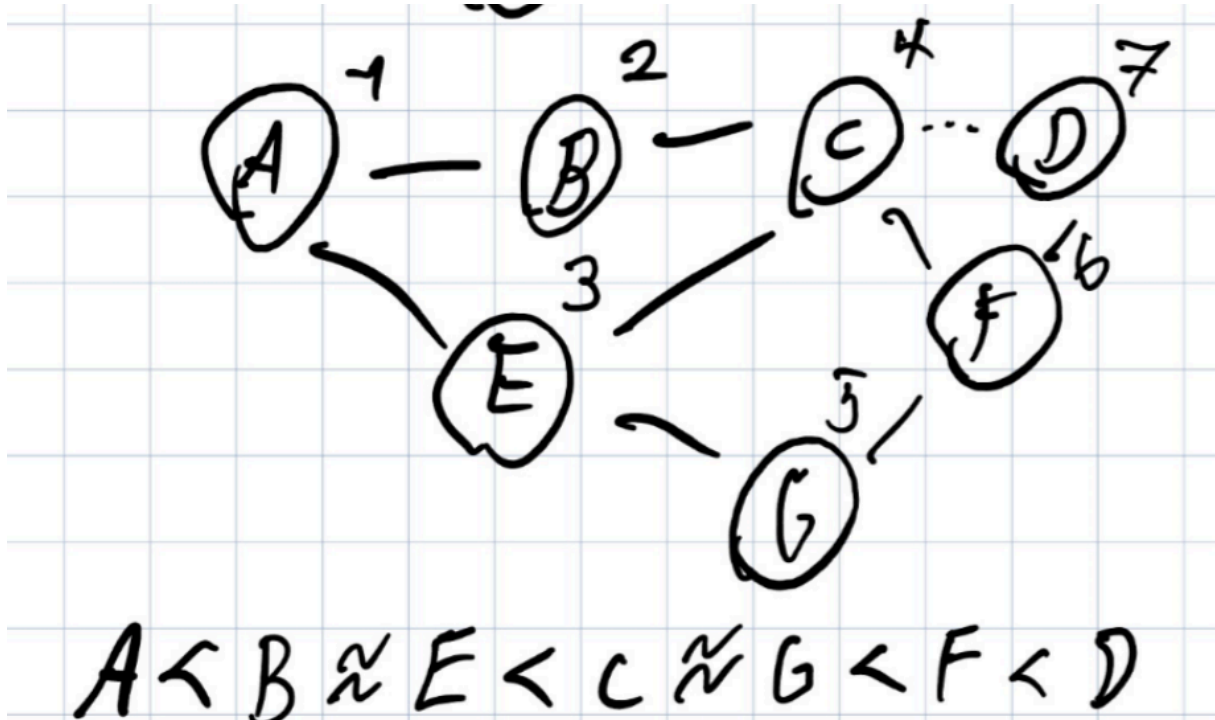
F: 0.8970627572016459

C: 1.7292074039995642



3.2 Schwierigkeit 1:

Da in Schwierigkeit 1 alle Gleichungen wahr sind, habe ich eine Zeichnung erstellt. (18.09.2024).



Vergleicht man diese Zeichnung mit dem Ergebnis, kann man erkennen, dass alle Aussagen durchgesetzt wurden und die eine ähnliche Struktur beibehalten hat.

$A < G < C < F < D$

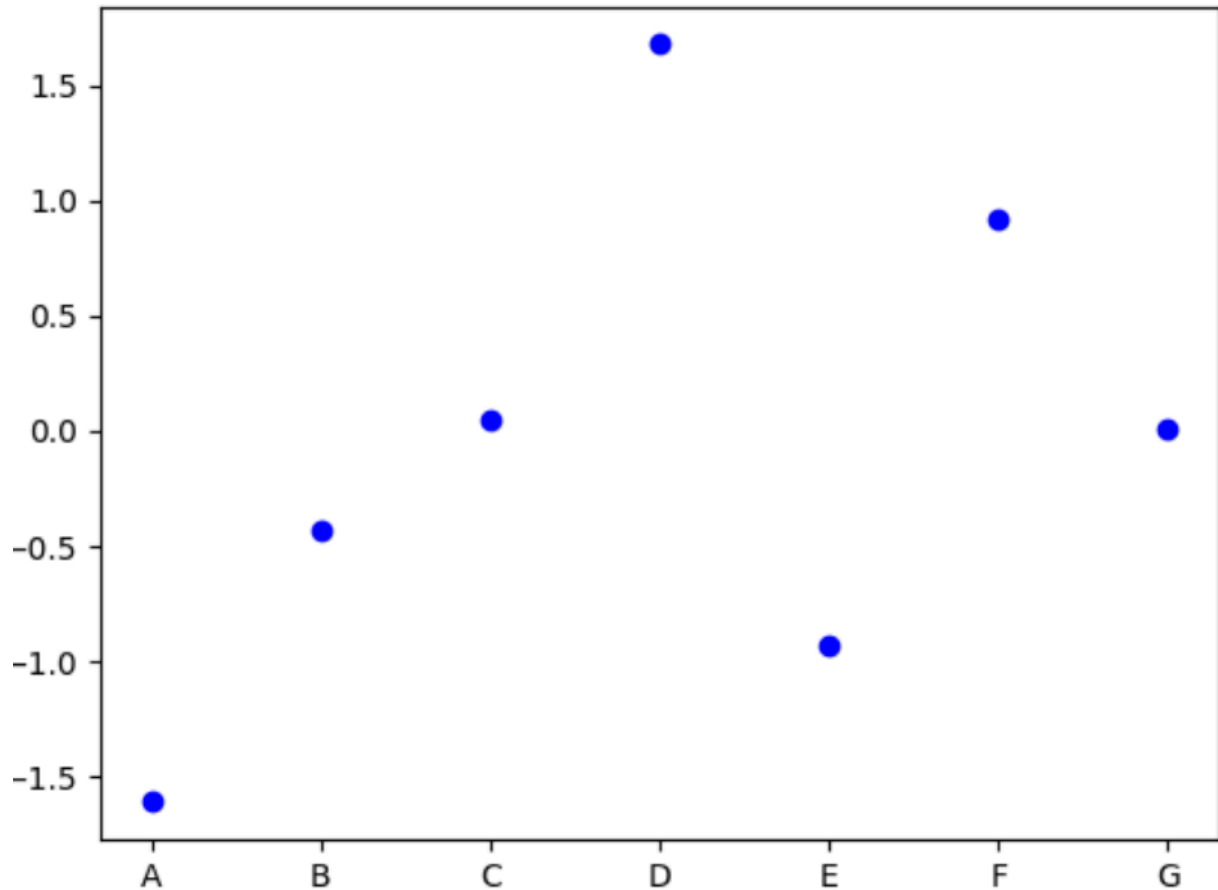
A: -1.6102109053497944

G: 0.008560385230909906

C: 0.05064443301326027

F: 0.9197861693783976

D: 1.6795784274500838



3.3 Schwierigkeit 2:

Zur Schwierigkeit 2 ist hier die letzte Gleichung: $D < E$ nennenswert. In Klausur 2 gilt, dass $E < D$ ist, jedoch gab es mehrere Prüfungen mit vielen Vergleichen, die belegt haben, dass E die einfachste Aufgabe ist. Dadurch konnte E trotzdem seine Position festhalten, jedoch fiel D aufgrund der letzten Gleichung zur zweit einfachsten Aufgabe.

$E < D < B < F < A < G$

E: -1.187844042924097

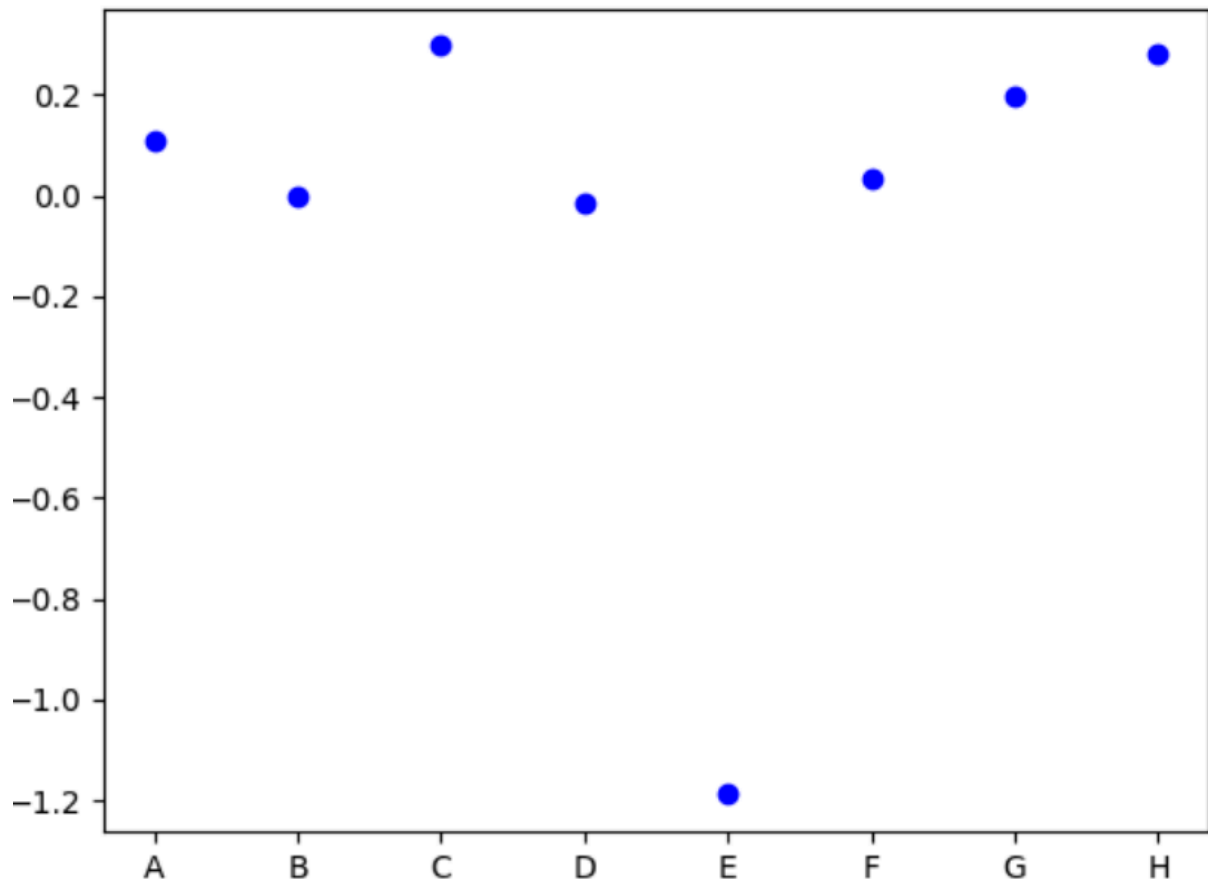
D: -0.015100308641975313

B: 0.0

F: 0.033692659512650436

A: 0.10738224451303158

G: 0.19535932617931717



3.4 Schwierigkeit 3:

In Aufgabe 3 ist die Gleichung $M < N$ und $N < M$ nennenswert. Beide Aufgaben kommen nur in der genannten Gleichung vor. In diesem Fall gilt die Regel, dass die neuere Prüfung einen größeren Wert haben sollte. Da $N < M$ die neuere Prüfung ist, hat sich es so durchgesetzt.

$C < I < B < D < A < H < L < N < M < J < E < K < F < G$

C: -1.3491907227604405

I: -1.3449382716049383

B: -1.2833306541495202

D: -0.7301922582304528

A: -0.5328703703703704

H: -0.19144032921810705

L: -0.1257407407407407

N: -0.040000000000000036

M: 0.040000000000000036

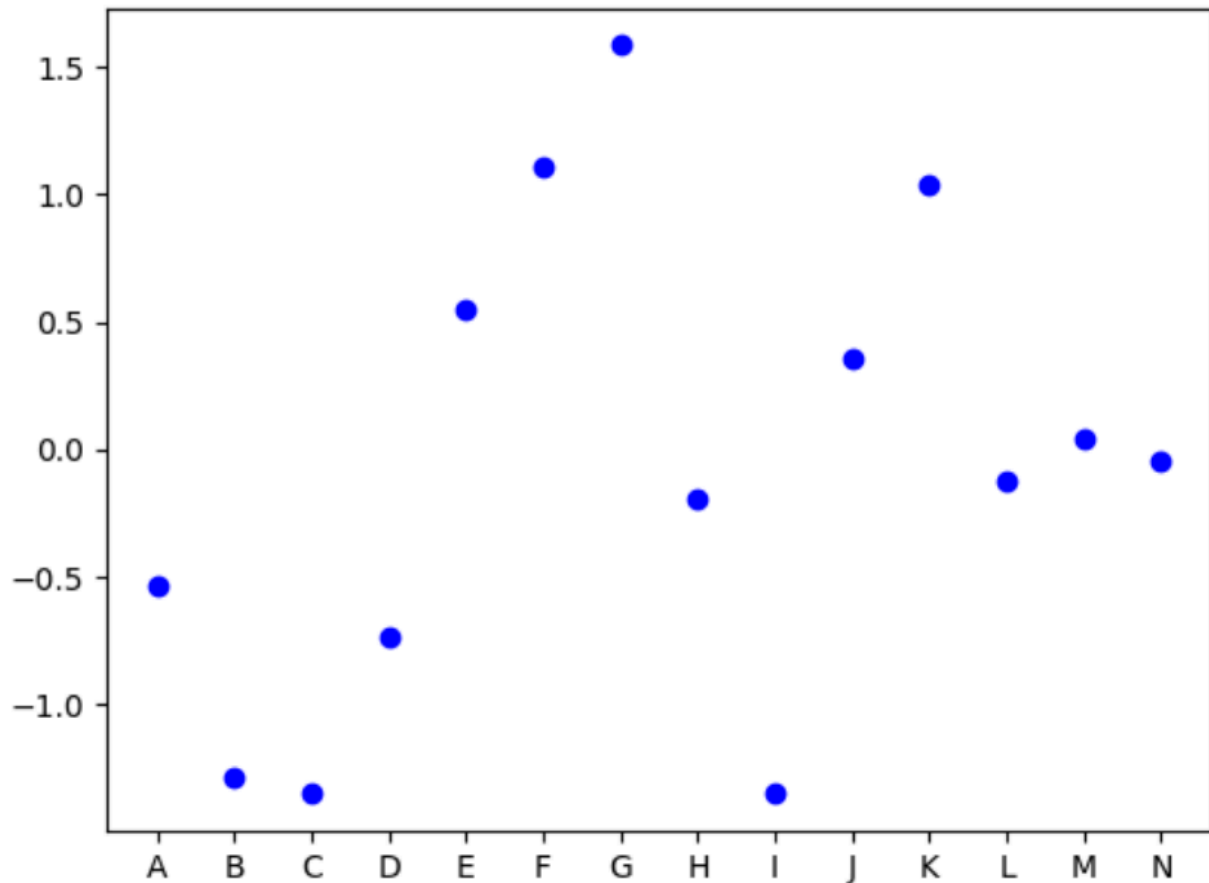
J: 0.3611111111111111

E: 0.5524144804526749

K: 1.0424382716049383

F: 1.107573112445543

G: 1.5856765474965708



3.5 Schwierigkeit 4:

In Schwierigkeit 4 wird deutlich, dass viele Prüfungen und Gleichungen einen Einfluss auf ein paar gewünschte Aufgaben haben. Dadurch wird deutlich, dass diese Lösungs idee keine Aufgaben (die nicht in den angeforderten Aufgaben enthalten sind) auslsst, sondern alle Aufgaben einen Einfluss haben.

$B < F < W < N < I$

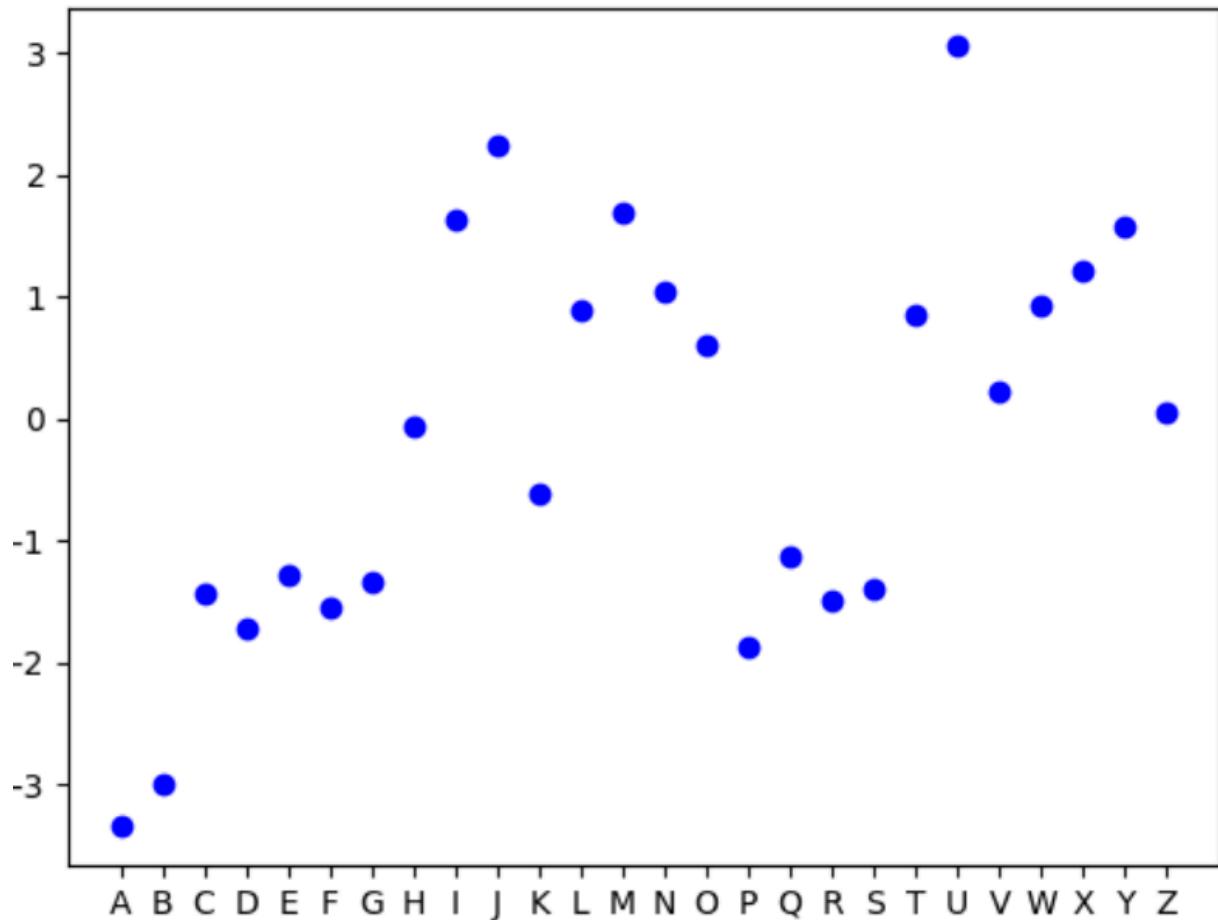
B: -2.9838173491301996

F: -1.5526406822262606

W: 0.9279415685408813

N: 1.0504952632045748

I: 1.644477329949905



3.6 Schwierigkeit 5:

Im letzten Beispiel wird es deutlich, dass diese Lösung auch funktioniert, wenn alle Aufgaben angefordert sind und gleichzeitig auch alle Aufgabenplätze belegt wurden.

$S < M < C < O < E < H < Z < R < Q < J < L < W < B < F < K < G < N < V < D < U < X < Y < I < P < T < A$

S: -2.655890296672582

M: -1.7623757701561904

C: -1.7191569703650746

O: -1.6131514013722077

E: -1.3520396255628817

H: -1.3442165352080477

Z: -1.2339750491746981

R: -0.9764848045798333

Q: -0.6873631919622204

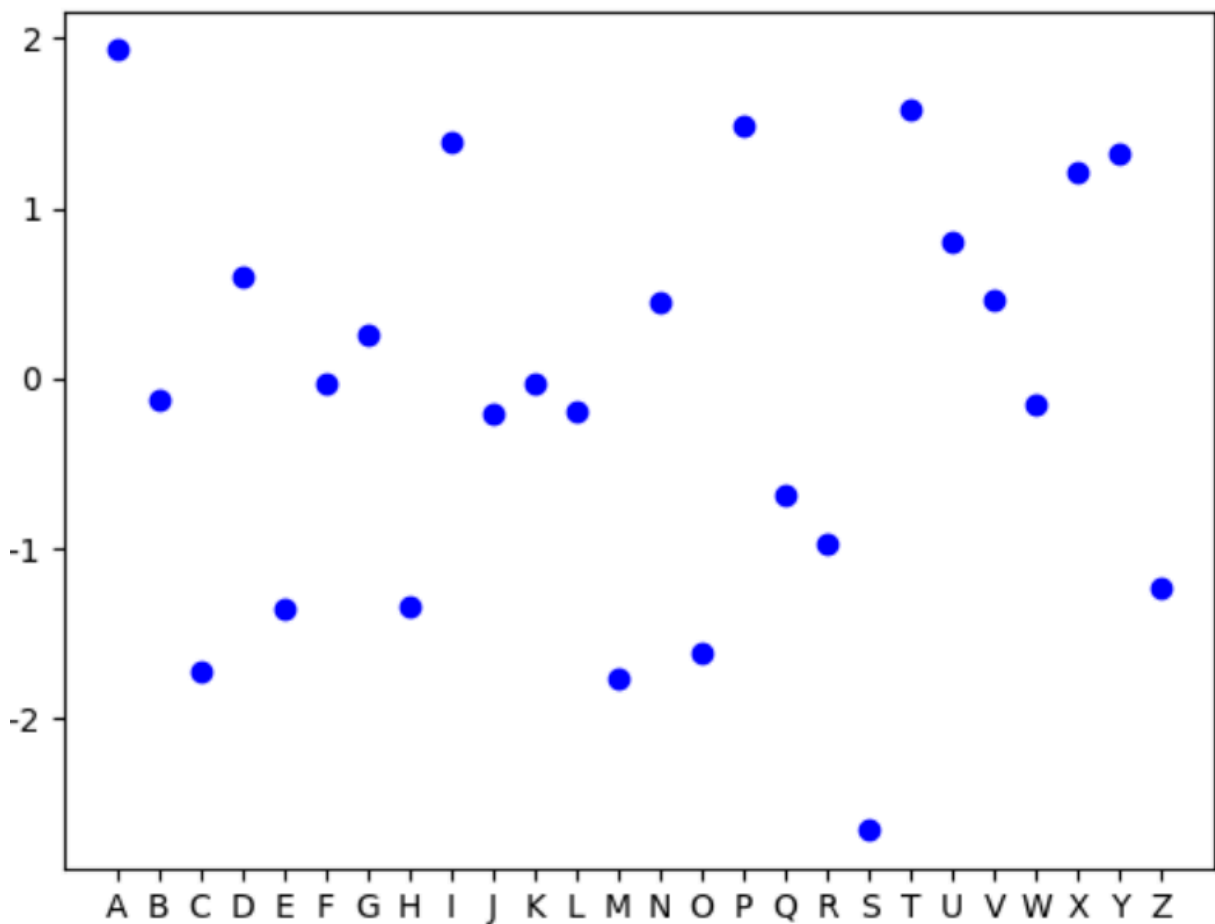
J: -0.20970831031219833

L: -0.1949218442163928

W: -0.15672320809641999

B: -0.1252784155928764

F: -0.026645279224219942

K: -0.025883267701222235 G: 0.2601555280128003 N: 0.45536423198957415 V: 0.45918684681895516 D: 0.5974614275036061 U: 0.8070468353515194 X: 1.2205928520491154 Y: 1.3216335792874316 I: 1.3967316308245912 P: 1.489796172953624 T: 1.5810934699978354 A: 1.9324027648987583 

4 Quellcode (Python 3.11.9)

```
def mathfunction(x):
    # vgl. Geogebra Grafik
    if x <= 0.1: return 0.5
    if x < 2.5:
        return 0.5*(-1/3*(x-0.1)+1)
    else:
        return 0.1
```

```

def diff(lower, higher) -> float:
    factor = 1
    delta = abs(lower - higher)
    output = mathfunction(delta)
    # Erwartet: Gleichung entspricht nicht die Werteverhältnis und sind mind. 0.5
    # Wertenheiten entfernt
    expected = not ((delta >= 0.5) and (lower > higher))
    if not expected:
        factor += 0.2
    # Unerwartete Ergebnisse Umrechnung
    if not expected and delta < 0.9:
        output = 1.3/3
    elif not expected and delta >= 0.9:
        output = 0.8 - output
    return output * factor

def main(m,k,examsHistory,k_var):
    Letters = "ABCDEFGHGIJKLMNOPQRSTUVWXYZ"
    tasks = {}
    for i in range(m):
        tasks.update({Letters[i]:0.0})
    del Letters

    # Die Schwierigkeitsgradwerte ausrechnen
    # Für alle vergangenen Prüfungen
    for exams in examsHistory:
        pendingTasksUpdates = tasks.copy()
        # Für alle Distanzen
        for distance in range(1,len(exams)):
            # Für einzelne Gleichungen
            for pointer in range(len(exams) - distance):
                pointerLetter = exams[pointer]
                targetLetter = exams[pointer + distance]
                pendingTasksUpdates[pointerLetter] -= diff(tasks[pointerLetter],
tasks[targetLetter])
                pendingTasksUpdates[targetLetter] += diff(tasks[pointerLetter],
tasks[targetLetter])
            # Nachdem alle Veränderungen für eine Distanz ausgerechnet wurden, Ergebnis
            # speichern.
            tasks = pendingTasksUpdates

    # Gewünschte Prüfungen sortieren und ausgeben
    sorted_tasks = sorted(tasks.keys(), key=tasks.get)
    targetKeys = [key for key in sorted_tasks if key in k_var]
    print(targetKeys[0], end="")
    for i in range(k-1):
        # Ausnahmefall: zwei Gleichungen haben die gleiche Wert
        if tasks[targetKeys[i]] == tasks[targetKeys[i + 1]]:
            print(" = ", end=targetKeys[i + 1])
        else:
            print(" < ", end=targetKeys[i + 1])
    print()
    # Tatsächliche Schwierigkeitswert ausdrucken.
    for key in targetKeys:
        print(f"{key}: {tasks[key]}")

```